BRAC University (Department of Computer Science and Engineering)
CSE 470 (Software Engineering) | Summer 2025

Quiz 3

Student ID:
Section: 18 - Set A **Full Marks: 20**

Name: **Duration: 25 minutes**

---

1. Twitter launches "MiniTube Alerts." Whenever the **Twitter** channel uploads a video, all subscribed users must be notified. Users should be able to **subscribe/unsubscribe** easily. New notification types (e.g., push, email) may be added later without touching the channel's code.
   a) Identify the design pattern. 2
   b)Provide **pseudocode** for the main classes (`Channel`, `User`, methods like `subscribe()`, `unsubscribe()`, `notify()`). 8
   c) State two benefits of this design. 2

Solution:

```
from typing import List, Dict


# --- Observer Pattern Interface ---
class Observer:
    def update(self, event: dict) -> None:
        raise NotImplementedError


class Subject:
    def subscribe(self, obs: Observer) -> None:
        raise NotImplementedError

    def unsubscribe(self, obs: Observer) -> None:
        raise NotImplementedError

    def notify(self, event: dict) -> None:
        raise NotImplementedError


# --- Concrete Subject: Channel (Twitter-only channel in MiniTube) ---
class Channel(Subject):
    def __init__(self, name: str):
        self._name = name
        self._observers: List[Observer] = []

    def subscribe(self, obs: Observer) -> None:
        if obs not in self._observers:
            self._observers.append(obs)

    def unsubscribe(self, obs: Observer) -> None:
```

```
        if obs in self._observers:
            self._observers.remove(obs)

    def notify(self, event: dict) -> None:
        for obs in list(self._observers):
            obs.update(event)

    def upload_video(self, title: str, url: str) -> None:
        event = {"type": "VIDEO_UPLOADED", "channel": self._name, "title": title, "url": url}
        self.notify(event)

    def start_live(self, topic: str) -> None:
        event = {"type": "LIVE_STARTED", "channel": self._name, "topic": topic}
        self.notify(event)


# --- One User Observer Class (handles all types of notifications) ---
class UserObserver(Observer):
    def __init__(self, user_id: str, preferences: Dict[str, bool]):
        self.user_id = user_id
        self.preferences = preferences  # Example: {'email': True, 'sms': False, 'in_app': True}

    def update(self, event: dict) -> None:
        if self.preferences.get('in_app', False):
            print(f"[InApp] -> {self.user_id}: {event['type']} from {event['channel']}")

        if self.preferences.get('email', False):
            print(f"[Email] -> {self.user_id}: '{event.get('title') or event.get('topic')}' now available.")

        if self.preferences.get('sms', False):
            print(f"[SMS] -> {self.user_id}: {event['type']} @ {event['channel']}")
```

2.Short Questions (8 marks)

a. (2 marks) In the **Singleton** pattern, what problem does it solve and give one common real-world example?
b. (2 marks) In the Observer Pattern, what is the role of the `Subject`?
c. (4 marks) Which pattern (Observer, Adapter, or Singleton) would you use if you want to:

i. Ensure only one database connection manager exists?
ii. Notify multiple users when an event occurs?

Solution:

a) **Singleton — intent + example:** Ensure exactly one instance of a class and provide a global access point. Example: a single app-wide **configuration** loader or **logger**.

b) **Observer — role of Subject:** Holds state/events; manages **subscribe/unsubscribe** and **notify** calls to observers when its state changes.

c) **Adapter — what stays vs. what changes:**

- Stays stable: the **target interface** used by your app (e.g., `VideoIngestor`).
- Changes behind the scenes: the **adapter** translating to a different **adaptee** API (legacy SDK).

d) **Pick the pattern for each goal:**

- i. Only one database connection manager → **Singleton**
- ii. Notify many listeners about one event → **Observer**